

LABORATOR 6

Comunicarea între procese: Socket

1. Scopul lucrării

Se urmărește studierea noțiunilor legate de **socket** și familiarizarea cu apelurile sistem utilizate în comunicarea prin socket-uri între procese. Lucrarea prezintă un model de comunicare de tip **client/server**.

2. Considerații teoretice

2.1. Introducerea noțiunii de socket

Sistemul de operare folosește TDF (Tabela **D**escriptorilor de **F**ișier) pentru a păstra pointeri la structuri interne pentru fișierele cu care lucrează procesul. Aplicația (procesul) folosește acești descriptori pentru accesarea unui fișier. Socket-urile, servesc la comunicarea în rețea, și asemeni descriptorilor de fișier sunt păstrați tot în TDF. Crearea unui socket se face prin apelul funcției sistem *socket(...)*. La acest apel sistemul de operare alocă o nouă structură de date ce păstrează informațiile necesare la comunicare și actualizează pointerul în TDF la această structură. Înainte ca socket-ul să poată fi utilizat în aplicații câmpurile structurii trebuie actualizate conform cerințelor aplicației prin alte apeluri sistem.

Un socket poate fi *pasiv* dacă este folosit de un proces server în vederea așteptării unor cereri de conectare, sau *activ* dacă este folosit de un proces client pentru a iniția o conexiune.

Cele mai importante informații din structura sunt adresa IP și numărul de port pentru protocol. Protocoalele TCP/IP definesc **punctul de comunicație** ca perechea adresa IP și numărul de port pentru protocol. Alte familii de protocoale definesc adresa punctului de comunicație în alte moduri. Deoarece socketul este utilizabil cu multe familii de protocoale el nu specifică modul de definire a adresei punctului de comunicație și nici formatul unei adrese pentru un protocol particular.

Pentru ca familiile de protocoale să aibă libertatea de a alege o reprezentare pentru adrese, socket-ul definește o **familie de adrese** pentru fiecare tip de adresa. Protocoalele TCP/IP folosesc o reprezentare unică pentru adrese, familia ardeselor fiind specificată prin constanta simbolică AF_INET (nu familia de protocoale PF_INET, deși ambele valori sunt identice).

În practică, programele de aplicație folosesc o structură predefinită cu două câmpuri: *identificatorul familiei de adrese* (2 octeți) și *adresa* (14 octeți). Din rațiuni de portabilitate, în cazul protocoalelor TCP/IP se recomandă utilizarea ei numai pe post de structură de păstrare adresa (singura referire este la câmpul *sa_family*). O adresa TCP/IP pentru un punct de comunicație este definită prin:

```
struct sockaddr_in {  
    u_short sin_family; // tip adresa (AF_INET)  
    u_short sin_port; // număr port pentru protocol  
    u_long sin_addr; // adresa IP ( declarata pe unele sisteme  
                        // ca struct in_addr)
```

```
char sin_zero[8]; // neutilizați  
};
```

2.2. Apelurile sistem pentru lucrul cu socket -uri

Complexitatea acestor apeluri este datorată numărului mare de argumente care însă le conferă o diversitate în utilizare. Un socket poate fi utilizat de un proces server sau client, pentru transferul datelor ca stream (TCP) sau datagrama (UDP), cu o adresă anume respectiv independenta de punctul de comunicație (client/server).

Principalele apeluri sunt:

- **socket** creează un nou descriptor de socket. Argumentele din apel specifică familia de protocoale folosite de aplicație (PF_INET pentru TCP/IP) și tipul serviciului (stream sau datagrama). Pentru un socket care folosește familia de protocoale Internet, tipul serviciului specifică dacă socket-ul folosește TCP sau UDP.
- **bind** leagă o adresa IP locală și portul protocolului la socket. Argumentele specifică descriptorul de socket și adresa punctului de comunicație. Pentru protocoalele TCP/IP specificarea adresei se face folosind structura *sockaddr_in*, ce include adresa IP și numărul de port al protocolului. Procesele server folosesc *bind* pentru a informa procesele client asupra portului unde se acceptă conexiuni.
- **listen** pentru server orientat pe conexiune, socket-ul devine pasiv și poate accepta cereri de conectare. De regula, codul ce implementează un proces server conține o buclă infinită în care acesta acceptă cererea de conectare, o tratează și revine la acceptarea unei noi cereri. Cu toate că timpul pentru tratarea unei cereri e foarte mic, o eventuala cerere sosită în acest interval poate fi pierdută. Din acest motiv primul argument precizează socket-ul, iar al doilea dimensiunea cozii de așteptare atașate socket-ului.
- **connect** stabilește o conexiune activă cu un server la distanță (apelat de procese client!). Argumentele precizează descriptorul de socket, adresa IP a host-ului pe care rulează serverul împreună cu numărul de port pentru protocol și dimensiunea adresei.
- **accept** pentru server creează un nou socket pentru noua cerere de conectare. Noul socket este folosit numai pentru noua conexiune, cel original servind în continuarea pentru acceptarea unor viitoare cereri de conectare. După ce datele au fost transferate folosind noul socket acesta este închis.
- **write** transmite date prin conexiunea realizată. Datele sunt, de regulă, copiate în buffer-urile din nucleul SO, aplicația își continuă execuția în timp ce datele sunt transportate prin rețea. Apelul este folosit de server pentru a transmite răspunsuri, iar de clienți pentru a transmite cereri.
- **read** citește datele ajunse la socket și le depune într-un tampon utilizator. Dacă nu sunt disponibile date apelul se blochează. Procesele client și server pot folosi *read* pentru a citi mesaje de la socket de tip UDP. Fiecare apel citește o datagrama. În cazul în care buffer-ul e prea mic surplusul este pierdut.
- **close** eliberează un socket. Dacă mai multe procese partajează un socket la fiecare *close* se decrementează un contor, iar când acesta ajunge la zero socket-ul este eliberat.

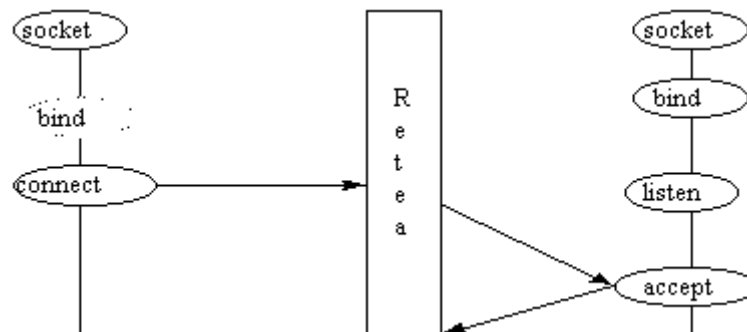


Fig. 1 Succesiunea apelurilor sistem la procesele client/server la folosirea unui socket de tip TCP

După cum se observa din figura procesul client creează socket-ul, se conectează la server (*bind* e opțional) după care trimite cereri de conectare folosind *write* și citește răspunsurile folosind *read*. La terminare conexiunea se poate închide prin *close*. Procesul server folosește *bind* pentru a specifica portul pentru protocol și apelează *listen* pentru a fixa dimensiunea cozii de așteptare. După aceasta el intra într-o buclă infinită unde folosind *accept* așteaptă o nouă cerere de conectare și folosește *read* și *write* pentru a interacționa cu clientul; în final închide conexiunea cu *close*. Procesul server revine la apelul *accept* în așteptarea unei noi cereri de conectare.

TCP/IP specifică o reprezentare standard pentru întregii binari folosiți în antetele de protocol. Reprezentare e numită **network byte order**, octetul mai semnificativ fiind primul reprezentat. De acest aspect trebuie ținut cont deoarece anumite rutine de lucru cu socket necesită argumentele în aceasta reprezentare. De exemplu, în structura *sockaddr_in* câmpul *sin_port*. Din rațiuni de portabilitate se recomandă întotdeauna folosirea rutinelor de conversie. Acestea sunt:

- **htons** și **ntohs** pentru conversia unui întreg scurt de la reprezentarea gazdă (**host**) la reprezentarea rețea (**network**) și invers;
- **htonl** și **ntohl** pentru conversia unui întreg lung de la reprezentarea gazdă (**host**) la reprezentarea rețea (**network**) și invers;

Tipul serviciului se poate specifica prin constantele simbolice: *SOCK_DGRAM* sau *SOCK_STREAM*.

Pentru detalii în utilizarea acestor apeluri sistem a se consulta exemplele.

2.3. Implementare

Implementarea modelului de comunicare client/server folosind socket-uri consta din două părți: implementarea procesului client și implementarea procesului server. În general, implementarea clientului este mai simplă, deoarece nu necesită drepturi speciale la folosirea porturi privilegiate, nu exista interacțiuni concurente cu mai multe servere și nu necesită protecție.

Clientul poate localiza server-ul (adică adresa IP și numărul de port pentru protocol) prin mai multe moduri (fiecare cu avantaje și dezavantaje). Uzual aceste informații se obțin dintr-un fișier de pe disc.

Implementarea procesului client astfel încât acesta să aibă ca argument în linia de comanda adresa serverului oferă acestuia independența față de locația server-ului. Un astfel de program poate fi combinat cu ușurință cu alte programe care extrag adresa server-ului fie dintr-un fișier, fie folosind un *nameserver* sau căutându-l printr-un protocol de broadcast.

Unele servicii implică un server anume (de exemplu, *remote login*) altele nu (de exemplu, *time*). Metoda de localizare a server-ului este și funcție de aplicație.

Clientul specifică adresa unui server prin structura *sockaddr_in*. Acest lucru înseamnă conversia adresei din reprezentarea cu punct (os.obs.utcluj.ro sau 193.226.6.154) la adresa IP (binară) pe 32 de biți. Interfața de lucru cu socket din BSD Unix conține două rutine de bibliotecă care realizează conversia: *inet_addr* (șir ASCII cu punct la adresa IP binară) și *gethostbyname* (șir ASCII cu domeniul hostului la pointer la structura *hostent*). Structura *hostent* e definită în fișierul antet *netdb.h* și conține câmpurile:

```
struct hostent {
    char *h_name; // nume host
    char **h_aliases; // lista alias
    int h_addrtype; // tip adresa
    int h_length; // dimensiune adresa
    char **h_addr_list; // lista adreselor
};
```

Pentru compatibilitate cu versiunile anterioară este definit câmpul *h_addr* ca fiind *h_addr_list[0]*.

Pentru a găsi portul unui protocol pentru un serviciu anume se poate folosi funcția de bibliotecă *getservbyname(...)*. Funcția are două argumente: serviciul solicitat și protocolul folosit. Funcția returnează un pointer la structura *servent* definită în *netdb.h*:

```
struct servent {
    char *s_name; // nume serviciu
    char **s_aliases; // lista alias
    int s_port; // portul pentru serviciul
    char *s_proto; // protocolul folosit
};
```

Exemplu: *getservbyname("echo", "tcp");*

Observație: Funcția *getservbyname* întoarce portul în formatul cerut în structura *sockaddr_in*.

Pentru a mapa numele unui protocol la valoarea atribuită protocolului se folosește funcția *getprotobyname(...)*. Valoarea întoarsa de funcție este zero dacă protocolul nu exista sau nu se poate citi fișierul cu mapări sau, în caz de succes, un pointer la structură:

```
struct protoent {
    char *p_name; // nume protocol
    char **p_aliases; // lista de alias
    int p_proto; // număr protocol
};
```

2.4 Aplicație client-server UDP

Aplicația implementează un schimb de date UDP între client și server.

```

//UDPServer.c

/*
 * gcc -o server UDPServer.c
 * ./server
 */
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#define BUFLen 512
#define PORT 9930

void err(char *str)
{
    perror(str);
    exit(1);
}

int main(void)
{
    struct sockaddr_in my_addr, cli_addr;
    int sockfd, i;
    socklen_t slen=sizeof(cli_addr);
    char buf[BUFLen];

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP))== -1)
        err("socket");
    else
        printf("Server : Socket() successful\n");

    bzero(&my_addr, sizeof(my_addr));
    my_addr.sin_family = AF_INET;
    my_addr.sin_port = htons(PORT);
    my_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    if (bind(sockfd, (struct sockaddr* ) &my_addr, sizeof(my_addr))== -1)
        err("bind");
    else
        printf("Server : bind() successful\n");

    while(1)
    {
        if (recvfrom(sockfd, buf, BUFLen, 0, (struct sockaddr*)&cli_addr, &slen)== -1)
            err("recvfrom()");
        printf("Received packet from %s:%d\nData: %s\n\n",
            inet_ntoa(cli_addr.sin_addr), ntohs(cli_addr.sin_port), buf);
    }

    close(sockfd);
    return 0;
}

```

```

//UDPClient.c

/*
 * gcc -o client UDPClient.c
 * ./client
 */

#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#define BUFLLEN 512
#define PORT 9930

void err(char *s)
{
    perror(s);
    exit(1);
}

int main(int argc, char** argv)
{
    struct sockaddr_in serv_addr;
    int sockfd, i, slen=sizeof(serv_addr);
    char buf[BUFLLEN];

    if(argc != 2)
    {
        printf("Usage : %s <Server-IP>\n",argv[0]);
        exit(0);
    }

    if ((sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP))==-1)
        err("socket");

    bzero(&serv_addr, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    if (inet_aton(argv[1], &serv_addr.sin_addr)==0)
    {
        fprintf(stderr, "inet_aton() failed\n");
        exit(1);
    }

    while(1)
    {
        printf("\nEnter data to send(Type exit and press enter to exit) : ");
        scanf("%s",buf);
        getchar();
        if(strcmp(buf,"exit") == 0)
            exit(0);
    }
}

```

```

        if (sendto(sockfd, buf, BUFLen, 0, (struct sockaddr*)&serv_addr, slen)==-1)
            err("sendto()");
    }

    close(sockfd);
    return 0;
}

```

2.5. Aplicații client-server TCP

Aplicația server-client următoare implementează funcția ecou, datele transmise de client către server sunt retransmise de acesta către client. Serverul rămâne în continuare în așteptarea unei noi conexiuni cu următoarele date recepționate de la client.

Aplicația client se execută cu doi parametri: adresa IP a serverului și portul utilizat. De exemplu: **./client 127.0.0.1 8080**

Server:

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define PORT 8081
int main()
{ struct sockaddr_in server;
  struct sockaddr_in from;
  char tampon[100];
  int sd;
  if ((sd=socket(AF_INET, SOCK_STREAM, 0))==-1)
  {
      printf("eroare socket()\n");
      exit (1);
  }
  bzero(&server, sizeof(server));
  bzero(&from, sizeof(from));
  server.sin_family=AF_INET;
  server.sin_addr.s_addr=htons(INADDR_ANY);
  server.sin_port=htons(PORT);
  if (bind(sd, (struct sockaddr*)&server, sizeof(struct sockaddr))==-1)
  {
      printf("eroare la bind(). \n");
      exit (1);
  }
  if (listen(sd, 5)==-1)
  {
      printf("eroare listen(). \n");
      exit (1);
  }

  while(1)
  {

```

```

int client;
int length=sizeof(from);
printf("se asteapta la portul %d ... \n",PORT);
fflush(stdout);
client=accept(sd, (struct sockaddr*)&from,&length);
if(client<0)
{
printf("eroare la accept(). \n");
continue;
}
bzero(tampon,100);
printf("asteapta mesajul... \n");
fflush(stdout);
if(read(client,tampon,100)<=0)
{
printf("eroare la read() de la client. \n");
close(client);
continue;
}
printf("mesaj receptionat... \n"
      "trimit mesaj inapoi...\n");
if(write(client,tampon,100)<=0)
{
printf("eroare la write() catre client. \n");
continue;
}
else
printf("transmitere cu success. \n");
close(client);
}
}

```

Client:

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <netdb.h>

int port;
int main(int argc, char*argv[])
{
int sd;
struct sockaddr_in server;
char tampon[100];
if(argc!=3)
{
printf("sintaxa:%s<adresa_server> <port> \n", argv[0]);
return -1;
}
port=atoi(argv[2]);
if((sd=socket(AF_INET,SOCK_STREAM,0))===-1)

```



```

{
printf("eroare la socket(). \n");
exit (1);
}
server.sin_family=AF_INET;
server.sin_addr.s_addr=inet_addr(argv[1]);
server.sin_port=htons(port);
if(connect(sd, (struct sockaddr*)&server, sizeof(struct sockaddr))== -1)
{
printf("eroare la connect(). \n");
exit (2);
}
bzero(tampon,100);
printf("introduceti mesajul:");
fflush(stdout);
read(0,tampon,100);
if(write(sd,tampon,100)<=0)
{
printf("eroare la write() spre server. \n");
exit (3);
}
if (read(sd,tampon,100)<0)
{
printf("eroare la read() de la server. \n");
exit (4);
}
printf("mesajul primit este:%s \n", tampon);
close(sd);
}

```